

Laboratorio 1: introducción al modelado y cálculo simbólico con SymPy

Optimización y Análisis de Redes

Víctor Aceña Gil / Antonio Alonso Ayuso

2026-06-16

Table of contents

1	Introducción	1
2	Configuración del entorno e instalación	2
3	Diferenciación simbólica y numérica en Python	2
3.1	Funciones univariadas	2
3.1.1	Enfoque numérico con Autograd	3
3.1.2	Enfoque simbólico con SymPy	3
3.2	Funciones multivariadas	3
3.2.1	Implementación con Autograd	4
3.2.2	Implementación con SymPy	4
3.3	Funciones paramétricas con SymPy	4
4	Estudio de curvatura y convexidad	5
5	Ejercicios de laboratorio propuestos	6
5.1	Ejercicio 1: gradiente y hessiana locales	6
5.2	Ejercicio 2: estudio de convexidad multivariable constante	6
5.3	Ejercicio 3: convexidad de funciones no cuadráticas (dominio factible)	7
5.4	Ejercicio 4: convexidad constante en 3D	7

1 Introducción

El modelado y la resolución de problemas de optimización matemática requiere una sólida comprensión de las propiedades analíticas de las funciones, en especial de su **curvatura** y

convexidad.

En este laboratorio práctico aprenderemos a utilizar Python para automatizar el análisis matemático local y global de funciones univariantes y multivariantes. Trabajaremos con dos herramientas fundamentales y complementarias:

1. **SymPy**: Una biblioteca de computación simbólica que nos permite realizar cálculos exactos (analíticos) de derivadas, gradientes, matrices hessianas y resolver sistemas de ecuaciones algebraicas de forma exacta.
2. **Autograd**: Una biblioteca de diferenciación automática que calcula de manera eficiente y con precisión de máquina las derivadas de funciones definidas mediante código de Python ordinario (cálculo numérico).

2 Configuración del entorno e instalación

Para asegurar que dispones de las librerías necesarias en tu entorno de Python, puedes instalarlas ejecutando:

```
pip install autograd sympy numpy
```

A continuación, importamos los módulos que utilizaremos:

```
import numpy as np
from autograd import grad, hessian
import sympy as sp

# Configurar SymPy para que imprima las expresiones matemáticas con formato elegante
sp.init_printing(use_unicode=True)
```

3 Diferenciación simbólica y numérica en Python

3.1 Funciones univariantes

Consideremos la función real de una variable:

$$f(x) = 5x + 10$$

Deseamos calcular su derivada (gradiente) en el punto de referencia $x_0 = 1.0$.

3.1.1 Enfoque numérico con Autograd

Autograd realiza diferenciación automática a partir de la definición estándar de la función en Python:

```
def f_num(x):
    return 5*x + 10

# Obtener la función derivada de primer orden
gradiente_num = grad(f_num)

x0 = 1.0
print("El gradiente numérico en x0 es:", gradiente_num(x0))
```

3.1.2 Enfoque simbólico con SymPy

SymPy construye un árbol de expresión analítica sobre una variable simbólica para realizar derivación formal:

```
# 1. Definir la variable simbólica
x = sp.symbols('x', real=True)

# 2. Definir la expresión matemática
f_sym = 5*x + 10

# 3. Calcular la derivada analítica
derivada_sym = sp.diff(f_sym, x)
print("Derivada analítica:", derivada_sym)

# 4. Evaluar la derivada en el punto x0
x0_val = 1.0
derivada_en_x0 = derivada_sym.subs(x, x0_val)
print("El gradiente simbólico evaluado en x0 es:", derivada_en_x0)
```

3.2 Funciones multivariantes

Analicemos ahora una función lineal multivariable (un plano en $\mathbb{R}^2$):

$$f(x_1, x_2) = 5x_1 + 10x_2 + 4$$

Deseamos evaluar en el punto $\mathbf{x}_0 = (1, 2)^T$: - El vector gradiente: $\nabla f(\mathbf{x}_0)$ - La matriz hessiana: $\mathbf{H}_f(\mathbf{x}_0)$

3.2.1 Implementación con Autograd

```
def plano_num(x):  
    return 5*x[0] + 10*x[1] + 4  
  
x0 = np.array([1.0, 2.0])  
  
# Obtener gradiente y hessiano  
grad_plano = grad(plano_num)  
hess_plano = hessian(plano_num)  
  
print("Gradiente numérico en x0:", grad_plano(x0))  
print("Hessiana numérica en x0:\n", hess_plano(x0))
```

3.2.2 Implementación con SymPy

```
# 1. Definir variables simbólicas  
x1, x2 = sp.symbols('x1 x2', real=True)  
  
# 2. Definir la función  
f_plano_sym = 5*x1 + 10*x2 + 4  
  
# 3. Calcular gradiente (como matriz columna de derivadas parciales)  
grad_plano_sym = sp.Matrix([sp.diff(f_plano_sym, var) for var in (x1, x2)])  
  
# 4. Calcular matriz hessiana  
hess_plano_sym = sp.hessian(f_plano_sym, (x1, x2))  
  
# 5. Evaluar en x0 = (1, 2)  
eval_dict = {x1: 1.0, x2: 2.0}  
print("Gradiente simbólico en x0:", grad_plano_sym.subs(eval_dict))  
print("Hessiana simbólica en x0:\n", hess_plano_sym.subs(eval_dict))
```

3.3 Funciones paramétricas con SymPy

Una gran ventaja del cálculo simbólico es la capacidad de trabajar con parámetros sin asignarles valores de antemano. Consideremos el plano con coeficientes genéricos a y b :

$$f(x_1, x_2) = ax_1 + bx_2 + 4$$

```

# Definir variables y parámetros simbólicos
x1, x2, a, b = sp.symbols('x1 x2 a b', real=True)

f_param = a*x1 + b*x2 + 4

# Calcular gradiente y hessiano analíticos genéricos
grad_param = sp.Matrix([sp.diff(f_param, var) for var in (x1, x2)])
hess_param = sp.hessian(f_param, (x1, x2))

print("Gradiente analítico genérico:\n", grad_param)
print("\nHessiana analítica genérica:\n", hess_param)

# Evaluar en el punto x0 = (1, 2) con parámetros a = 5, b = 10
res_grad = grad_param.subs({x1: 1, x2: 2, a: 5, b: 10})
print("\nGradiente sustituyendo parámetros:", res_grad)

```

4 Estudio de curvatura y convexidad

Para evaluar si una función es convexa, cóncava o indefinida en un dominio, se estudia la definición de su matriz hessiana:

- **Convexa:** La matriz hessiana es semidefinida positiva ($\mathbf{H}_f(\mathbf{x}) \succeq 0$) para todo $\mathbf{x}$. Sus autovalores son no negativos ($\lambda_i \geq 0$).
- **Estrictamente convexa:** La matriz hessiana es definida positiva ($\mathbf{H}_f(\mathbf{x}) \succ 0$) para todo $\mathbf{x}$. Sus autovalores son estrictamente positivos ($\lambda_i > 0$).
- **Cóncava:** La matriz hessiana es semidefinida negativa ($\mathbf{H}_f(\mathbf{x}) \preceq 0$) para todo $\mathbf{x}$. Sus autovalores son no positivos ($\lambda_i \leq 0$).

Podemos utilizar NumPy para hallar numéricamente los autovalores a partir del hessiano simbólico convertido:

```

# Definir una función auxiliar para obtener autovalores numéricos
def calcular_autovalores(matriz_sympy, punto_dict=None):
    if punto_dict:
        mat_eval = matriz_sympy.subs(punto_dict)
    else:
        mat_eval = matriz_sympy
    mat_np = np.array(mat_eval).astype(np.float64)
    return np.linalg.eigvals(mat_np)

```

5 Ejercicios de laboratorio propuestos

Resuelve de forma interactiva en tu cuaderno los siguientes ejercicios aplicando las herramientas que acabamos de describir.

5.1 Ejercicio 1: gradiente y hessiana locales

Dada la función de segundo grado:

$$f(\mathbf{x}) = 2x_1 + 6x_2 - 2x_1^2 - 3x_2^2 + 4x_1x_2$$

Calcula su vector gradiente y su matriz hessiana en el punto de origen $\mathbf{x}_0 = (0, 0)^T$ de dos maneras:

1. Empleando **Autograd**.
2. Empleando **SymPy**.

```
# Escribe tu solución aquí:  
# ...
```

5.2 Ejercicio 2: estudio de convexidad multivariable constante

Estudia analíticamente la convexidad de la siguiente función cuadrática en todo su dominio:

$$f(\mathbf{x}) = x_1x_2 + 2x_1^2 + x_2^2 - 6x_1x_3$$

Para ello:

1. Define simbólicamente la función en SymPy.
2. Obtén la matriz hessiana simbólica.
3. Convierte la matriz a NumPy y calcula sus autovalores.
4. Con base en el signo de los autovalores, concluye si la función es convexa, cóncava o indefinida.

```
# Escribe tu solución aquí:  
# ...
```

5.3 Ejercicio 3: convexidad de funciones no cuadráticas (dominio factible)

Estudia la convexidad de la función:

$$f(\mathbf{x}) = 10 - 2(x_2 - x_1^2)^2$$

1. Calcula la matriz hessiana simbólica en términos de las variables x_1 y x_2 .
2. Dado que los elementos de la matriz dependen de las variables, la convexidad varía en distintas regiones del espacio. Encuentra analíticamente la región o conjunto de puntos (x_1, x_2) donde la función es **cóncava** (es decir, donde el menor principal de primer orden y el determinante alternan de signo: $M_1 \leq 0$ y $M_2 \geq 0$).
3. Utiliza la función `sp.solve` para hallar la región correspondiente de la variable x_2 en función de x_1 .

```
# Escribe tu solución aquí:  
# ...
```

5.4 Ejercicio 4: convexidad constante en 3D

Determina si la función:

$$f(\mathbf{x}) = x^2 + 3y^2 + z^2 + xy - 4z - 5x + 14$$

es estrictamente convexa en su dominio calculando los autovalores de su matriz hessiana con SymPy y NumPy.

```
# Escribe tu solución aquí:  
# ...
```